

On this page

Migrate Cassandra Profile Store

This document provides a step-by-step guide for migrating the Cassandra Profile Store, which stores the scheduled metrics (SMJs) values.

Please note this page outlines one of the many approaches to migrate a keyspace/table. This migration is not mandatory but reduces the possibility of metric collisions between channels.

Migration Overview

Migrating the Profile Store from a single keyspace/table to multiple keyspaces/tables requires several steps to ensure that the metrics are not unavailable during the migration process. The process includes the following steps:

1. Prepare the new Cassandra keyspace and table by creating all the necessary data structures
2. Backfill the new table with the existing data to avoid null/zero metrics
3. Update the Pulse properties to start using the new keyspace/table
4. Clean up the old keyspace/table

Migration Steps

In this section you can find a description of the migration steps.

For this migration process explanation, we will assume that the objective is to migrate the table `profiles` in the keyspace `profiles` to a new table `uk_oub_profiles` in a new keyspace `uk_profiles`.

Prepare the new Cassandra keyspace and/or table

The first step is to prepare the new keyspace and/or table, ensuring it has the same definition as the existing one.

This can be done by following the next steps:

1. Describe the existing keyspace/table

```
DESCRIBE profiles;
```

Copy

2. Copy the output from the previous command, e.g.

```
CREATE KEYSPACE profiles WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1': '1'} AND durable_writes = true;

CREATE TABLE profiles.profiles (
  key text,
  column1 text,
  value text,
  PRIMARY KEY (key, column1)
) WITH CLUSTERING ORDER BY (column1 ASC)
AND additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.1
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.LeveledCompactionStrategy', 'max threshold': '32', 'min threshold': '4'}
```

```

AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compression.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';

```

Copy

3. Rename the keyspace/table and create the new keyspace/table

```

CREATE KEYSPACE uk_profiles WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1': '1'} AND durable_writes = true;

CREATE TABLE uk_profiles.uk_oub_profiles (
    key text,
    column1 text,
    value text,
    PRIMARY KEY (key, column1)
) WITH CLUSTERING ORDER BY (column1 ASC)
AND additional_write_policy = '99p'
AND bloom_filter_fp_chance = 0.1
AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
AND cdc = false
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.LevelledCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compression.LZ4Compressor'}
AND memtable = 'default'
AND crc_check_chance = 1.0
AND default_time_to_live = 0
AND extensions = {}
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair = 'BLOCKING'
AND speculative_retry = '99p';

```

Copy



Please note the commands above have default configuration values (e.g. replication factor) which might have to be adjusted according to the cluster configuration.

Backfill data

To ensure that we have consistent metrics values while migrating the profile store keyspace and/or table, we should backfill it with the current data. Backfilling is an essential step to guarantee that metrics are always available in the current/new Pulse cluster.

This can be done using the CQL `COPY` command to export the current data and import it into the new table. Example:

```

COPY profiles.profiles TO 'profiles.csv';
COPY uk_profiles.uk_oub_profiles FROM 'profiles.csv';

```

Copy

Please refer to the official Cassandra documentation for more details regarding the CQL `COPY` command:

- [COPY TO](#)
- [COPY FROM](#)

Update Pulse properties

To complete the migration, some Pulse properties need to be redirected to the new keyspace/table:

- Keyspace name (Ansible variable: `cassandra_pulse_keyspace`):

```
pulse.app.{appSlug}.services.rte.workflow.plan.scheduledMetrics.profileStore.cassandra.keyspace.name
```

Copy

- Table name (Ansible variable: `cassandra_table_name`):

```
pulse.app.{appSlug}.services.rte.workflow.plan.scheduledMetrics.profileStore.table
```

Copy

The properties can be updated through Pulse UI or by using Ansible variables (requires a deploy). When updated through the Pulse UI, a publish with restart is required but all changes will be lost once a Blue/Green deployment is performed.

The application variable override should be used to set different keyspaces/tables for different applications. For example, considering a single UK keyspace and different tables per application, the configuration would be:

```
cassandra_pulse_keyspace : "uk_profiles"
uk_outbound_payments_cassandra_table_name: "uk_oub_profiles"
uk_cards_cassandra_table_name : "uk_cds_profiles"
uk_inbound_payments_cassandra_table_name : "uk_inb_profiles"
```

Copy

After these changes are done and all applications are published, the migration process is now complete.

Clean-up

Once validated that everything is working as expected, the old keyspace/table can be safely removed:

```
DROP TABLE profiles.profiles;
DROP KEYSPACE profiles;
```

Copy